Graph Tokens before Graph Propagation: Atom-Keyed Retrieval Interfaces for Graph Foundation Models

Anonymous Authors[1]

Abstract

Graph foundation models for language-facing applications must decide not only how to propagate over a graph, but also what object should become a graph token and how that token should connect to the text an LLM reads. We study this interface in long single-document QA. Our position is that the unit a graph retriever *ranks* and the unit a reader *consumes* should be decoupled by a deterministic source map. Atom-Keyed Chunk Retrieval (AKCR) treats atomic Davidsonian propositions as graph nodes, uses typed edge hints such as entity overlap, scene index, and temporal anchors as schema primitives, ranks atoms by query similarity, and maps selected atoms back to source chunks as the LLM payload.

On full-book NarrativeQA (40 books, 1,169 questions), AKCR gives a bounded but useful efficiency result. With the same GLM-4.7 reader, AKCR at 24K characters scores 0.5543, above a chunk-cosine retriever at the same budget (0.5227), while remaining below the full-book oracle (0.6450). With GLM-5.1, retrieval exceeds the degraded full-context oracle, but we read this as long-context degradation rather than as evidence that retrieval beats context. A scoring-rule ablation favors atom-best max pooling, while two negative results set the boundary of the claim: a learned typed-edge propagator drives $\mathrm{ppr}_\alpha \to 0.95$, effectively disabling propagation in this single-book pool, and feeding atoms directly to the reader plateaus near 0.46. We therefore argue for *graph-token first* evaluation protocols for GFM–LLM interfaces: test the token, source-map, and payload design before attributing gains to propagation.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Graph foundation models (GFMs) promise transferable graph representations across schemas, label spaces, and downstream tasks (Mao et al., 2024; Jiang et al., 2024). Recent graph–LLM systems ask models to reason over textual graphs, graph computational problems, or retrieved graph context (He et al., 2024; Chen et al., 2024b). For language-facing GFMs, however, a more basic interface decision appears before the choice of backbone: *what object should become the graph token, and how should that token be grounded back to the text an LLM reads?* The answer matters because graph-side operations and language-side reasoning often want different units. A graph retriever wants compact, typed, comparable nodes; an LLM reader wants coherent natural-language evidence.

Standard retrieval-augmented generation (RAG) (Karpukhin et al., 2020) treats a chunk, a contiguous span of text, as both the retrieval unit and the reader unit. Graph-augmented variants (Edge et al., 2024; Gutiérrez et al., 2024) add entity or relation graphs on top of chunks and then propagate through that graph with PageRank-like or learned operators. These systems have improved retrieval coverage, but they still leave open a graph-foundation question: should the same object serve as the graph node, ranking key, and language payload? We study a simple alternative in which these roles are separated by design.

Our testbed is long single-document question answering, a setting where the answer often depends on a small narrative fact buried inside a much larger text. We represent each chunk by atomic Davidsonian propositions (Davidson, 1967). Each atom is a graph node with entity mentions, a scene index, a time anchor, and typed edge hints such as entity overlap, shared subject, scene co-occurrence, and inferred temporal or causal links. The source-of relation $\sigma : t \to c$ maps an atom t to its source chunk c, forming a deterministic bipartite map between the graph-token layer and the language-payload layer. Atom-Keyed Chunk Retrieval (AKCR) ranks atoms, collapses them to source chunks through σ, and feeds those chunks to the reader. Figure 1 shows the complete interface.

This design turns a broad GFM question into a measurable

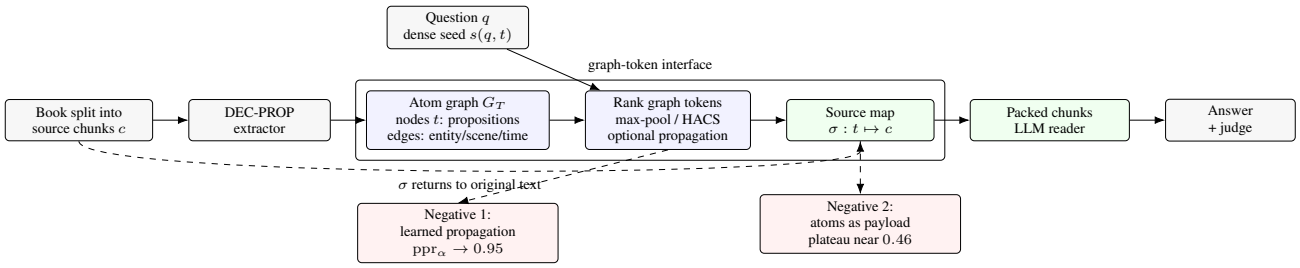

Figure 1. AKCR as a graph-token interface for language-facing GFMs. The graph layer ranks atomic proposition tokens and may use typed structural hints; the payload layer remains the original source chunks. The deterministic source map σ is the key interface object: it lets retrieval use fine-grained graph tokens while the reader consumes coherent text. The two diagnostic failures isolate where the design boundary lies in our current setting.

one. If the atom is the graph token, does typed propagation over atom edges improve on a strong dense atom seed? If the atom is an accurate proposition, can it also serve as the reader payload, or must the reader return to the original source chunk? These questions are small enough to evaluate carefully, but they target core GFM issues: graph tokenization, structural hints, graph–LLM interfaces, and evaluation protocols that reveal when graph structure helps.

We evaluate AKCR on full-book NarrativeQA (Kočiský et al., 2018): 40 books, 4,929 chunks, and 1,169 questions. A 3-pass GLM-4.7 extraction pipeline produces 89,575 atomic propositions. AKCR ranks atoms against the query in the BGE-M3 embedding space (Chen et al., 2024a), maps selected atoms back to source chunks, and asks an LLM reader to answer from the packed chunks. With the same GLM-4.7 reader, HACS-AKCR at a 24K-character budget scores 0.5543, above a chunk-cosine retriever at the same budget (0.5227), while remaining below the full-book oracle (0.6450). Thus, the matched-reader result is an efficiency gain over chunk retrieval, not a claim that retrieval beats reliable full context. With GLM-5.1, retrieval exceeds the degraded 78K-character full-context oracle; we treat this as long-context degradation, not as evidence in favor of retrieval over context.

The negative results are as important as the positive ones. A scoring-rule ablation favors atom-best max pooling over sum, log-sum-exp, and atom-prepending variants. A learned typed-edge propagator (LearnRet, APPNP closed form (Klicpera et al., 2019), 16 trainable parameters) drives $\mathrm{ppr}_\alpha \to 0.95$, effectively disabling propagation in this single-book pool. Feeding atoms directly to the reader plateaus near 0.46 despite richer extraction formats. Together, these results suggest that in this regime the token/source-map/payload interface explains more of the gain than graph propagation itself.

Contributions. We make five contributions for graph-foundation evaluation with LLM readers. First, we formulate AKCR as a graph-token interface: atoms are ranked graph tokens, chunks are language payloads, and σ is an explicit source map. Second, we report matched-reader retrieval and oracle comparisons on full-book NarrativeQA, showing a GLM-4.7 gain over chunk-cosine retrieval at the same budget while staying below the reliable full-context oracle. Third, we give a scoring-rule ablation showing that atom-best max pooling is the most stable reduction among the variants we tested. Fourth, we report two useful negative results, learned propagation self-disable and atoms-as-content failure, that mark where graph structure or graph-side payloads did not help. Fifth, we document non-trivial API run-to-run variance for LLM-judged QA and recommend multi-run reporting and variance-aware interpretation for GFM–LLM retrieval benchmarks.

2. Related Work

Graph foundation models and graph–LLM systems. Recent GFM work asks how graph representations can transfer across tasks, schemas, and domains rather than being retrained per dataset (Mao et al., 2024). A parallel graph–LLM line studies retrieval-augmented graph reasoning, textual graph QA, and instruction following over graph computational tasks (He et al., 2024; Jiang et al., 2024; Chen et al., 2024b). Our work is closest to this interface line, but it focuses on a lower-level design choice: what should serve as the graph token when the downstream reader is an LLM?

Retrieval units for long-context readers. Dense passage retrieval uses chunks as both ranking keys and evidence units (Karpukhin et al., 2020), while multi-vector retrieval shows that finer retrieval keys can improve ranking without forcing the reader to consume token-level fragments (Santhanam et al., 2022). GraphRAG, HippoRAG, and RAPTOR introduce graph or hierarchy structure over textual evidence (Edge et al., 2024; Gutiérrez et al., 2024; Sarthi et al., 2024). AKCR differs by making the source map explicit: atomic propositions are graph tokens for ranking, but source chunks remain the language payload.

Atomic propositions and decontextualization. Decontextualized sentences can make facts stand alone (Choi et al., 2021), and Davidsonian event-style representations provide a compact proposition format (Davidson, 1967). Our negative atoms-as-content result shows why such units should not automatically become reader inputs: a proposition can be a useful graph token while still being a poor narrative payload.

3. A Graph-Token Interface View

Thesis (bounded). *For long single-document narrative QA, our evidence suggests that the choice of graph token is a useful design choice to study before adding propagation. In our setting, using atomic Davidsonian propositions as ranking keys and source chunks as reader payloads, joined by a deterministic source-of map, gives a better budget–accuracy trade than chunk-cosine retrieval at the same budget, and recovers a substantial fraction of full-context oracle accuracy without exceeding it under a matched, well-behaved reader. We do not generalise to multi-hop encyclopedic QA, schema-grounded KGQA, or non-narrative long documents.*

Why role-decoupling can help here. A long document of C chunks admits two retrieval geometries: one d-dimensional vector per chunk (chunk-RAG), or roughly 17 vectors per chunk, one per atomic proposition (atom-RAG). Under the former, the chunk's score is the *average* of its propositions: one relevant fact buried in twenty unrelated ones is ranked at the average. Under the latter, the chunk's score is the *best* of its propositions, because we rank atoms and only afterwards collapse to source chunks via σ. The geometry is reminiscent of ColBERT-v2 multi-vector retrieval (Santhanam et al., 2022), with the difference that our multi-vectors are LLM-emitted Davidsonian propositions (typed by predicate, anchored by time and scene), not transformer token vectors.

Why both endpoints can fail. A chunk is a coarse graph token: it averages multiple propositions and discourse roles. A flat atom list is a coarse reader payload: decontextualization (Choi et al., 2021) strips the narrative voice the reader composes from. We treat the atom as the graph token, the source chunk as the language payload, and σ as the deterministic bipartite map between them. We do not argue this is the right decomposition for graph-to-LLM interfaces in general; we argue it is a practical interface choice in the single-document long-context regime, and one whose failure cases (atoms-as-content; learned propagation on a saturated dense seed) are themselves informative about where graph-side operations might add value.

4. Empirical Evidence

4.1. Setup

We evaluate on full-book NarrativeQA (40 books, 4,929 GLM-4.7-segmented chunks of ∼800 chars, 1,169 questions). Atomic memory is extracted once, query-independently, by a 3-pass GLM-4.7 pipeline (DECPROP): (1) **Decontextualize**, replacing pronouns with antecedent surface forms; (2) **Davidsonian extract**, emitting atomic propositions with explicit `subject`, `predicate`, `object`, `time_anchor`, `scene_index`, `salience`, and `edge_hints`; (3) **Faithfulness + coverage critic**, dropping hallucinations and adding missing atoms. Output: 89,575 atoms (mean ∼17/chunk), ∼1.5× the original chunk char-count, ∼24 minutes at 200-way concurrency, one-time cost ∼¥150.

For each query, AKCR scores atoms against the query using a frozen BGE-M3 encoder, restricted to the source story; takes top-K atoms ($K = 200$); maps each atom to its source chunk; deduplicates while preserving atom rank; packs source chunks DOS-sorted by chunk index into the reader's character budget. Three readers are evaluated: Claude Opus 4.7, GLM-5.1, and GLM-4.7. The judge is GLM-4.7 in pairwise reference-grounded mode.

4.2. Same-reader retrieval quality and oracle gap

We organise the empirical evidence into three blocks: (1) same-reader comparisons against the full-context oracle and against a chunk-cosine retrieval baseline at matched budget; (2) a budget sweep against the chunk baseline under the GLM-5.1 reader; and (3) a cross-reader deployment-style note. Table 1 groups the same-reader rows (BLOCK A) and a chunk-cosine baseline (BLOCK B); the cross-reader observation is summarised in §4.2 and shown as a separate block (BLOCK C).

Block A: same-reader retrieval vs. full-context oracle. At $b = 24{,}000$ chars, $n = 1{,}169$, with GLM-4.7 as both retrieval reader and oracle reader, HACS-AKCR scores 0.5543 versus a 0.6450 full-book oracle: a gap of -9.07pp. On the $n = 149$ subset where we also have an Opus 4.7 oracle, AKCR scores 0.7338 versus 0.8322, a gap of -9.84pp. We do not claim AKCR closes this gap; it does not. AKCR recovers a fraction of full-context performance (about 86% of GLM-4.7 oracle, about 88% of Opus oracle on that subset) at ∼30% of the input cost. With the GLM-5.1 reader, the same-reader comparison reverses (0.5672 AKCR at 24K vs 0.4311 oracle at 78K), but we treat this as diagnostic of long-context degradation in GLM-5.1 (consistent with lost-in-the-middle behaviour, Liu et al. 2024), not as evidence that AKCR exceeds full context under a well-behaved reader.

Table 1. Empirical results on full-book NarrativeQA, $b = 24{,}000$ chars, $n = 1{,}169$ unless noted. BLOCK A reports same-reader retrieval vs. full-context oracle; BLOCK B reports same-reader AKCR vs. chunk-cosine retrieval at the same budget; BLOCK C is the cross-reader deployment note (§4.2). Same-reader AKCR remains below the same-reader oracle when the oracle is reliable (GLM-4.7, Opus); the GLM-5.1 oracle row is marked as diagnostic since it is degraded by long-context behaviour at 78K chars. "±" is the standard deviation across reruns where reported.

Method	Reader	judge_acc	gap to same-reader oracle
BLOCK A: SAME-READER RETRIEVAL VS. FULL-CONTEXT ORACLE			
Long-context oracle (78K)	GLM-4.7	0.6450	—
HACS-AKCR ($\alpha=0.8$, 24K)	GLM-4.7	0.5543	−9.07pp
Long-context oracle (78K, $n=149$ subset)	Opus 4.7	0.8322	—
HACS-AKCR (24K, $n=149$ subset)	Opus 4.7	0.7338	−9.84pp
Long-context oracle (78K) *(diagnostic)*	GLM-5.1	0.4311	—
AKCR v1 (24K, 4-rerun mean)	GLM-5.1	0.5552 ± 0.053	+12.4pp †
BLOCK B: SAME-READER RETRIEVAL VS. CHUNK-COSINE BASELINE			
DOS+chunks (chunk-cosine, 24K)	GLM-4.7	0.5227	—
HACS-AKCR ($\alpha=0.8$, 24K)	GLM-4.7	0.5543	—
DOS+chunks (24K)	GLM-5.1	0.5389	—
AKCR v1 (24K, 4-rerun mean)	GLM-5.1	0.5552 ± 0.053	—
Atoms-as-content (GAR V013, 24K)	GLM-5.1	0.4594	—
BLOCK C: CROSS-READER DEPLOYMENT OBSERVATION			
HACS-AKCR (24K, 3 reruns)	Opus 4.7	0.6721 ± 0.007	—
Long-context oracle (78K)	GLM-4.7	0.6450	—

† *Diagnostic only*: we read AKCR (24K) above the GLM-5.1 full-context oracle (78K) as a reflection of long-context degradation in GLM-5.1, not as a retrieval-vs-context win.

Block B: same-reader retrieval vs. chunk-cosine retrieval. At the same $b = 24$K budget, the chunk-cosine retrieval baseline (DOS+chunks: rank chunks by $\cos(q, c)$, pack into the budget) under GLM-4.7 scores 0.5227, below the AKCR same-reader score of 0.5543 (within run-to-run variance characterised in §4.4). Under GLM-5.1, an AKCR v1 multi-run sample (four reruns, mean 0.5552 ± 0.053) is consistent with, but not strictly above, the single-run DOS+chunks number (0.5389). We therefore claim a same-reader retrieval improvement at GLM-4.7, and a same-reader retrieval result that is above or at parity with chunk-cosine at GLM-5.1 within noise; we do not claim a robust same-reader win at GLM-5.1.

Block C: cross-reader deployment observation. With Claude Opus 4.7 as retrieval reader, HACS-AKCR at $b =$ 24K reaches 0.6721 ± 0.007 over three reruns. The GLM-4.7 long-context oracle on the full ∼78K-char book scores 0.6450. We report this only as a deployment-style data point: a stronger reader on ∼30% of the book producing comparable judge accuracy to a weaker reader on the full book. It is not evidence about retrieval vs. full context under a matched reader, and we do not treat it as a primary scientific claim. We further caution that this observation depends on the relative reading capability of Opus 4.7 vs. GLM-4.7 on this benchmark; with a different reader pair, the direction can plausibly invert.

Pareto across budgets. On a single-run budget sweep ($b \in \{2, 4, 8, 12, 16, 24\}$K chars, GLM-5.1 reader), AKCR v1 was above DOS+chunks at five of six budgets, with the largest gap +8.3pp at $b = 12$K and a tie at $b = 8$K. The associated single-run paired-bootstrap P-values look strong, but because the underlying run-to-run dispersion (§4.4) is comparable to a few of these gaps we treat the qualitative ordering, not the per-budget significance, as the take-away.

Table 2. Scoring-rule ablation: paired bootstrap vs v1 rerun #2 (May 8; $n = 1{,}169$, GLM-5.1 reader). HACS $\alpha = 0.8$ ties v1 within noise; alternatives degrade or tie. ACRP regresses (−7.3pp) because atoms-as-pointer pulls the reader toward atoms-as-content failure.

Rule	judge_acc	Δpp	95% CI	$P(> v1)$
v1 ($\alpha=1.0$)	0.5595	—	—	—
HACS $\alpha=0.8$	0.5672	+0.77	$[-1.71, +3.25]$	0.72
HACS $\alpha=0.7$	0.5423	−1.71	$[-4.28, +0.77]$	0.08
MAR-sum	0.5458	−1.37	$[-3.93, +1.11]$	0.13
MAR-lse $\tau=5$	0.5355	−2.40	$[-4.96, +0.17]$	0.03
ACRP $K_{\text{kf}}=15$	0.4867	−7.27	$[-10.0, -4.62]$	0.000

Cost note. DEC-PROP extraction is a one-time corpus-side cost (∼¥150 for $4{,}929$ chunks at our concurrency setting); BGE-M3 encoding runs on Apple MPS; per-query retrieval is a CPU dense matmul. Figure 2 visualizes the two main empirical diagnostics using values loaded directly from the experiment summary files.

4.3. Scoring-rule ablation

The atom-keyed retrieval admits a family of chunk-scoring rules. Define the top-K atom set T_q for query q and let $s(t) = \cos(q, t)$ be the dense atom-query score. We compare: (i) **v1**, atom-best: rank chunks by their highest-scoring atom in T_q; (ii) **MAR-sum**: chunk score $= \sum_{t \in c, t \in T_q} s(t)$; (iii) **MAR-lse** ($\tau = 5$): log-sum-exp of in-chunk top-K atom scores; (iv) **HACS** (α): $\alpha \cdot \max_{t \in c} s(t) + (1-\alpha) \cdot \cos(q, c)$; (v) **ACRP**: same chunks as v1, but the top-15 atoms are also prepended to the reader prompt as "key facts." Table 2 reports paired-bootstrap deltas vs a fresh v1 rerun ($n = 1{,}169$, GLM-5.1 reader, $b = 24{,}000$).

Reading the ablation. In our setting, HACS $\alpha = 0.8$ ties v1 (within run-to-run noise; §4.4); HACS $\alpha = 0.7$, MAR-sum, and MAR-lse all degrade weakly. ACRP regresses sharply: prepending atoms to the reader prompt appears to induce an atoms-as-content failure mode (the reader tends to paraphrase the atom list rather than synthesise from the chunks). Among the rules we tried, atom-best max-pooling is the only reduction that consistently survives. We do not claim it is uniquely optimal in general; it is the rule we keep recommending in this single-document setup until counter-evidence appears. HACS $\alpha = 0.8$ is also the variant that drives the cross-reader headline in Table 1: the 20% chunk-cosine bonus appears to recover scene-/discourse-level context without breaking the atom-dominated ranking.

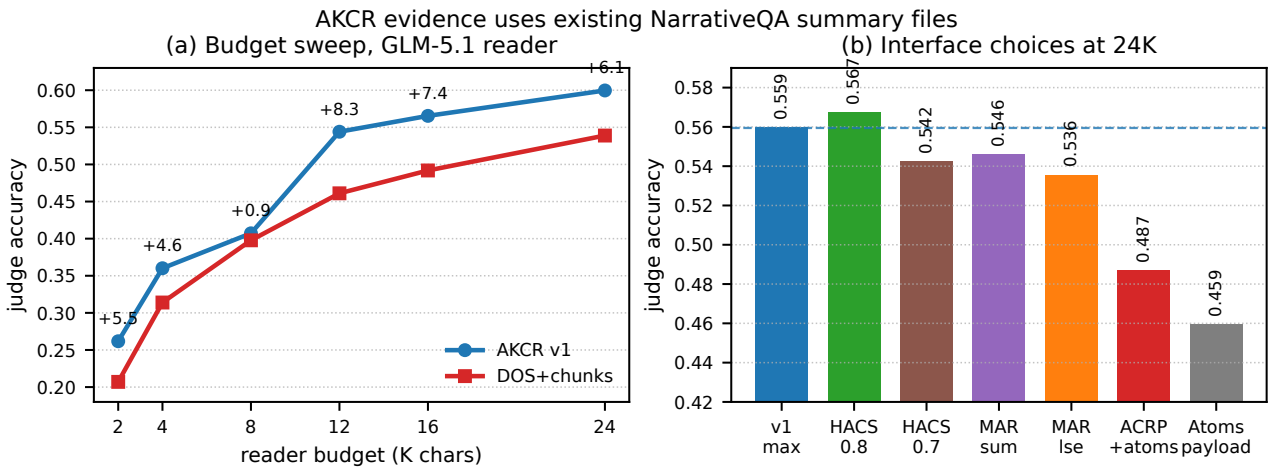

Figure 2. Experiment visualizations from existing NarrativeQA summary files. (a) In a single-run GLM-5.1 budget sweep, atom-keyed retrieval is above DOS+chunks at five of six budgets; we treat the ordering as qualitative because API run-to-run variance is non-trivial. (b) At 24K characters, max-style atom-to-chunk reductions are the most stable choices, while prepending atoms (ACRP) and feeding atoms as the reader payload both degrade accuracy. These panels support the interface claim: graph tokens help ranking, but source chunks remain the safer language payload.

4.4. Robustness and variance-aware interpretation

Across all numbers in Table 1 we apply the following interpretation rules.

(1) Run-to-run variance. Four identical-configuration AKCR v1 reruns ($n = 1{,}169$, $b = 24$K, `temperature=0`, GLM-5.1 reader) gave judge_acc $\in \{0.5997, 0.5595, 0.5825, 0.4790\}$, mean 0.5552 ± 0.053, range 0.121. Single-run paired-bootstrap confidence intervals are tighter than this dispersion, so we report multi-run means where feasible and treat sub-3pp single-run gaps as inconclusive.

(2) Comparison to chunk baseline across budgets. On the GLM-5.1 budget sweep, AKCR v1 was above DOS+chunks at five of six budgets. Several of those gaps are within run-to-run noise; we therefore make a qualitative claim (AKCR is at least at parity with chunk-cosine across budgets), not a strong significance claim per budget.

(3) Oracle-gap retention. Defining $\text{retained\_oracle\_fraction} = \text{AKCR}_{24\text{K}}/\text{Oracle}_{78\text{K}}$ under a matched reader, we obtain $\sim 0.5543/0.6450 \approx 0.86$ for GLM-4.7 and $\sim 0.7338/0.8322 \approx 0.88$ for Opus on the $n = 149$ subset, i.e., AKCR retains roughly 86%–88% of full-context accuracy at $\sim 30\%$ of input chars. Defining $\text{gap\_closed\_vs\_chunk} = (\text{AKCR}_{24\text{K}} - \text{Chunk}_{24\text{K}})/(\text{Oracle}_{78\text{K}} - \text{Chunk}_{24\text{K}})$, we obtain at GLM-4.7 roughly $(0.5543\text{–}0.5227)/(0.6450\text{–}0.5227) \approx 0.26$, i.e., AKCR at the matched 24K budget closes about a quarter of the gap between chunk-cosine retrieval and the full-context oracle. Both ratios are budget- and reader-dependent, so we report them as descriptive diagnostics rather than headline numbers.

(4) GLM-5.1 oracle is diagnostic. We mark the GLM-5.1 full-context oracle row in Table 1 as diagnostic: at 78K-char inputs the GLM-5.1 oracle is degraded relative to GLM-4.7 in a manner consistent with long-context behaviour reported in the literature (Liu et al., 2024). Same-reader AKCR exceeding the GLM-5.1 oracle should not be read as a general retrieval-vs-context result.

(5) Negative effect sizes. The two negatives in §5 have effect sizes ($|\Delta| \geq 2.4$pp on the relevant comparisons) larger than the per-run dispersion above, and the LearnRet $\text{ppr}_\alpha \to 0.95$ result is structural rather than a bootstrap effect.

5. Lessons from Negatives

We report two negative results that together bound the position. Both are negative outcomes for design choices a graph-foundation pipeline might try first; we read them as constraints on *when* graph-side propagation or graph-side payloads add value, not as evidence against propagation in general.

5.1. Learned propagation self-disables on saturated single-book seeds

We built a typed multi-edge graph over atoms (entity overlap, shared subject, scene index, and edge-hint relations) and ran Personalized PageRank with hand-tuned weights. Hand-tuned PPR lost 3.7pp F1@15 vs. dense-only on the same retrieval target. We then made all 16 graph hyperparameters trainable as `nn.Parameter`, used the APPNP

closed-form solve (Klicpera et al., 2019), and trained with a multi-positive InfoNCE loss (LearnRet). The optimizer drove $\mathrm{ppr}_\alpha \to 0.95 \pm 0.01$, which makes the propagation update reduce, in the limit, to the dense seed. A natural interpretation is that the dense atom seed is already sufficient when the candidate pool is restricted to a single book of $\sim$5,000 chunks: the structured graph adds little that the seed has not already captured. We do not claim propagation is useless in general; multi-hop encyclopedic QA, multi-document settings, and schema-grounded KGQA may differ. We claim that, in this single-book regime with this edge typology, the learned propagator did not find a non-trivial use of the graph.

5.2. Atoms-as-content caps at 0.46 regardless of format

We tested four atom-feed-to-reader variants: flat bullet list of V010 atoms, 0.307 at GLM-4.7; GAR V010 (graph format with edges), 0.458 at GLM-5.1; GAR V012 (richer extraction with verbatim sources and typed edges), 0.458; GAR V013 (verbatim source as displayed text plus modality annotations), 0.459. None breaks the 0.46 ceiling. Stronger extractors do not change the ceiling: V010 (GLM-4.7 extractor, mean lexical recall 0.859), V011 (GLM-5.1 extractor, 0.895), V012 (GLM-5.1 with verbatim+edges+modality, more selective). The bottleneck is not extraction quality; it is that decontextualized propositions lose the narrative voice the reader needs to compose answers. Hybrid retrieval (atoms + chunks + summaries) at $b = 24{,}000$ scores 0.519; adding RAPTOR cluster summaries does not exceed 0.431 at $b = 16{,}000$. None of these break 0.6.

6. Discussion

Implication for GFM design. Our results support a modest design principle for language-facing GFMs: evaluate the graph token and its source map before attributing gains to graph propagation. In our setup, once atomic proposition nodes give a dense retriever a clean answer-region seed, a learned typed-edge propagator finds no useful non-trivial update, while using the same atoms as reader payload damages answer synthesis. The useful object is therefore not just a graph, but a three-part interface: graph token, source map, and language payload.

Where this position is bounded. Our results are restricted to long-document QA over narrative and semi-narrative text in NarrativeQA. We do not claim the position generalises to multi-hop encyclopedic QA (e.g., HotpotQA, MuSiQue), where edge traversal can connect facts no single chunk contains; to schema-grounded KGQA; or to non-narrative long documents (legal, scientific, code). Each is a candidate stress test and a candidate falsifier. In particular, distractor-heavy multi-hop benchmarks such as HotpotQA are a natural future stress test for AKCR: when distractor paragraphs are crafted to share entities with the query, a max-pool over atom cosines is no longer obviously the right ranking signal, since high-similarity atoms may come from distractors. We frame that as a hypothesis to be tested, not as a reported result of this paper.

What a GFM benchmark should report. A graph–LLM retrieval benchmark should separate at least four axes: the graph tokenization (chunk, entity, event, proposition), the structural hints or edge schema, the source map from graph nodes to reader payloads, and the reader budget. Without this separation, a system can look like a graph propagation success when the actual gain comes from a better token, or look like a graph failure when the reader payload has been made unreadable. The AKCR protocol is one concrete way to expose these axes, and the two negatives show why the axes matter.

Open questions for the GFM community. (i) *What is the right tokenization granularity, and how should it scale with document length?* Atomic propositions work in our setup; sub-atomic fragments may overshoot, while chunks may blur too many facts. (ii) *When does graph propagation add value on top of a strong dense atomic seed?* Our learned propagator self-disabling is a falsification signal, not a general refutation; multi-hop, multi-document, or sparse-coverage regimes are the natural places to look for the boundary. (iii) *How should graph tokens transfer across schemas?* A proposition token may transfer across books because it has a stable predicate-argument form, but knowledge graphs, tables, scientific papers, and code repositories may require different node schemas and different source maps. This is where the broader graph-foundation question remains open.

7. Limitations

AKCR is an interface and evaluation protocol for graph-token retrieval, not a pre-trained universal GFM. We evaluate one benchmark family (NarrativeQA full-book), one atom-edge schema, and three commercial/API readers, so the generalisation to multi-hop QA, schema-grounded KGQA, non-narrative long documents, and open-weight readers remains untested. The DEC-PROP extractor and GLM-4.7 judge are also LLM components; we use a shared judge across conditions, but absolute scores remain judge-conditional.

8. Conclusion

Language-facing GFMs need more than a graph backbone: they need a clear interface between graph tokens and LLM-readable evidence. AKCR instantiates one such interface by

ranking atomic proposition nodes and mapping them back to source chunks through σ. On full-book NarrativeQA, this design improves over chunk-cosine retrieval at matched budget, but the negative results are equally important: learned propagation self-disables in this single-book pool, and atoms are poor reader payloads. The practical lesson is therefore graph-token first, but payload-aware: evaluate the token, source map, and reader budget before attributing gains to propagation.

Declaration on LLM Use

LLMs are used as pipeline components: **GLM-4.7** for DEC-PROP atomic extraction (3 passes), as one of three reader configurations, and as the shared pairwise reference-grounded judge across all conditions; **GLM-5.1** as a second reader configuration and as an alternative extractor in ablation variants (not as judge); **Claude Opus 4.7** as the third reader configuration. All three are commercial/API models accessed during the experimental period. The authors used automated tools for copy-editing and LaTeX formatting assistance. All experiments, numerical results, tables, claims, and interpretations were verified by the authors and remain their responsibility.

References

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024a.

Chen, N., Li, Y., Tang, J., and Li, J. GraphWiz: An instruction-following language model for graph computational problems. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 353–364, 2024b. doi: 10.1145/3637528.3672010.

Choi, E., Palomaki, J., Lamm, M., Kwiatkowski, T., Das, D., and Collins, M. Decontextualization: Making sentences stand-alone. *Transactions of the Association for Computational Linguistics (TACL)*, 9:447–461, 2021.

Davidson, D. The logical form of action sentences. In Rescher, N. (ed.), *The Logic of Decision and Action*. University of Pittsburgh Press, 1967.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. From local to global: A GraphRAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. HippoRAG: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

He, X., Tian, Y., Sun, Y., Chawla, N. V., Laurent, T., LeCun, Y., Bresson, X., and Hooi, B. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Jiang, X., Qiu, R., Xu, Y., Zhang, W., Zhu, Y., Zhang, R., Fang, Y., Chu, X., Zhao, J., and Wang, Y. RAGraph: A general retrieval-augmented graph learning framework. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. Dense passage retrieval for open-domain question answering. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

Klicpera, J., Bojchevski, A., and Günnemann, S. Predict then propagate: Graph neural networks meet personalized PageRank. In *International Conference on Learning Representations (ICLR)*, 2019.

Kočiský, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., and Grefenstette, E. The NarrativeQA reading comprehension challenge. *Transactions of the Association for Computational Linguistics (TACL)*, 6:317–328, 2018.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12: 157–173, 2024.

Mao, H., Chen, Z., Tang, W., Zhao, J., Ma, Y., Zhao, T., Shah, N., Galkin, M., and Tang, J. Graph foundation models. *arXiv preprint arXiv:2402.02216*, 2024.

Reimers, N. and Gurevych, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

Santhanam, K., Khattab, O., Saad-Falcon, J., Potts, C., and Zaharia, M. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2022.

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., and Manning, C. D. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.